

Systemprogrammierung Prüfung, WS25/26

Mathias Lampert

6. April 2026

Inhaltsverzeichnis

1	C-Sprache	2
1.1	Bytespeicher für <code>uint8_t*</code> und <code>uint16_t*</code>	2
1.2	Einsatz von <code>static</code> in C	2
1.3	Endianness	2
2	Konfiguration	3
3	Bitmaskierung	4
3.1	Aufgabe: <code>setCtrl</code> -Funktion	4
3.2	Aufgabe: Rotate Right	4
4	Dynamische Speicherverwaltung	5
4.1	Aufgabe: <code>mem_calcStatistics</code>	5
5	Multitasking	6
5.1	Semaphor	6
5.2	Semaphor in FreeRTOS	6
5.3	Producer/Consumer Implementierung	7
5.4	Scheduling Diagramm	8
5.5	Producer/Consumer Erweiterung	8
6	Probeklausur Systemprogrammierung 2024/2025	9
6.1	Aufgabe 1: Datentypen	9
6.2	Aufgabe 2: RGB-Farbe und Union	9
6.3	Aufgabe 3: Bitmaskierung	10
6.4	Aufgabe 4: Dynamische Speicherverwaltung	10
6.5	Aufgabe 5: Multitasking	11

1 C-Sprache

1.1 Bytespeicher für `uint8_t*` und `uint16_t*`

Wie viele Bytes werden zur Speicherung eines `uint8_t*` verwendet, wie viele für einen `uint16_t*`? Begründen Sie die Antwort:

Beide Pointer belegen den gleichen Speicherplatz. Die Größe eines Pointers hängt von der Systemarchitektur ab. Ein 32-Bit-System verwendet 4 Bytes für eine Adresse.

Ein 64-Bit-System verwendet 8 Bytes. Ein Pointer speichert ausschließlich die Speicheradresse. Der Datentyp des Ziels definiert die Interpretation der Daten. Der Datentyp ändert die Größe des Pointers nicht.

1.2 Einsatz von `static` in C

Wo kommt das Schlüsselwort `static` in C überall zum Einsatz? Geben sie Beispiele und erklären Sie die jeweilige Bedeutung:

- Lokale Variablen: Die Variable behält ihren Wert über das Funktionsende hinaus. Das System speichert sie im Datensegment statt auf dem Stack. Beispiel: `static int counter = 0;` in einer Funktion.
- Globale Variablen: Das System beschränkt den Zugriff auf die aktuelle Quelldatei. Andere Dateien sehen diese Variable nicht. Beispiel: `static int file_internal_status;`
- Funktionen: Die Funktion ist nur innerhalb der eigenen C-Datei aufrufbar. Das verhindert Namenskonflikte beim Linken. Beispiel: `static void helper(void);`

1.3 Endianness

Erleutern Sie den Begriff Endianness (Little Endian vs. Big Endian). Inwiefern kann die Endianness bei der Entwicklung und Portierung von Software in C eine Rolle spielen?

Endianness definiert die Byte-Reihenfolge im Speicher. Little Endian speichert das niederwertigste Byte an der kleinsten Adresse. RISC-V nutzt dieses Format. Big Endian speichert das höchstwertigste Byte zuerst. Portierungsprobleme treten auf, wenn Software Daten über Netzwerke schickt oder binäre Dateiformate zwischen Systemen austauscht. Sie müssen die Byte-Reihenfolge in diesen Fällen manuell umkehren.

2 Konfiguration

Ein Sensor-Modul liefert einen 32-Bit-Konfigurationswert, in mehrere Parameter (Mode, level, threshold, flags) jeweils in 8 Bit codiert sind.

Definieren sie eine struct SensorConfig, die vier Komponenten beinhaltet. Ebenso soll über die union SensorData auf die Komponenten oder den gesamten 32-Bit raw-Wert zugegriffen werden können.

Schreiben sie eine Funktion printSensorData() in C, welche einen Pointer auf eine SensorData erhält und sowohl den raw-Wert als auch die einzelnen Strukturkomponenten per printf() ausgibt.

```
#include <stdio.h>
#include <stdint.h>

typedef struct {
    uint8_t mode;
    uint8_t level;
    uint8_t threshold;
    uint8_t flags;
} SensorConfig;

typedef union {
    uint32_t raw;
    SensorConfig config;
} SensorData;

void printSensorData(SensorData *data) {
    printf("Raw-Wert: 0x%08X\n", data->raw);
    printf("Mode: %u, Level: %u, Threshold: %u, Flags: %u\n",
           data->config.mode,
           data->config.level,
           data->config.threshold,
           data->config.flags);
}
```

3 Bitmaskierung

3.1 Aufgabe: setCtrl-Funktion

Schreiben sie eine C-Funktion void setCtrl, die an den über pCtrl adressierten Bereich die gegebenen Flags schreibt, ohne die restlichen Bits zu beeinflussen. enableMotor ist Bit 0, enableLight ist Bit 8, enable Sensors ist Bit 16.

```
void setCtrl(uint32_t* pCtrl, bool enableMotor, bool enableLight,
            bool enableSensors) {
    uint32_t mask = (1 << 0) | (1 << 8) | (1 << 16);
    *pCtrl &= ~mask;
    if (enableMotor)    *pCtrl |= (1 << 0);
    if (enableLight)    *pCtrl |= (1 << 8);
    if (enableSensors) *pCtrl |= (1 << 16);
}
```

3.2 Aufgabe: Rotate Right

Implementieren sie in C eine Funktion, die einen 16-Bit-Wort um n Bit nach rechts rotiert (rotate Right). Die unteren n Bit sollen oben wieder eingefügt werden. Der Originalwert und n sind Parameter, der rotierte Wert ist Rückgabe. Beispielt mit 8 Bit: 01100101 -> rot 2 bit -> 01011001

```
uint16_t rotateRight(uint16_t value, uint8_t n) {
    n %= 16;
    return (value >> n) | (value << (16 - n));
}
```

4 Dynamische Speicherverwaltung

4.1 Aufgabe: mem_calcStatistics

In der dynamischen Speicherverwaltung der Übungen zur Lehrveranstaltung fehlt eine Funktion `mem_calcStatistics()`. Diese Funktion soll die durchschnittliche Größe (arithmetisches Mittel) der allokierten und freien Speicherblöcke zurückgeben. Dazu müssen alle Speicherblöcke durchwandert werden. Der Speicher, den die Header verbrauchen, soll nicht mitgezählt werden. Die Checksumme kann beim Durchlaufen ignoriert werden.

```
void mem_calcStatistics(float* avgAlloc, float* avgFree) {
    uint32_t sumAlloc = 0, countAlloc = 0;
    uint32_t sumFree = 0, countFree = 0;
    uint32_t offset = 0;

    while (offset < HEAPSIZE) {
        struct Header* h = (struct Header*)&gHeap[offset];
        if (h->flags & 0x01) {
            sumAlloc += h->length;
            countAlloc++;
        } else {
            sumFree += h->length;
            countFree++;
        }
        offset += sizeof(struct Header) + h->length;
    }
    *avgAlloc = countAlloc ? (float)sumAlloc / countAlloc : 0;
    *avgFree = countFree ? (float)sumFree / countFree : 0;
}
```

5 Multitasking

5.1 Semaphore

Was ist ein Semaphore? wie funktioniert er? Was bedeuten die Aufrufe P() und V()? Ein Semaphore steuert den Zugriff auf Ressourcen. Er nutzt einen Zähler.

- P() (Take): Verringert den Zähler. Ist der Zähler Null; wartet der Task.
- V() (Give): Erhöht den Zähler. Das System weckt einen wartenden Task auf.

5.2 Semaphore in FreeRTOS

Erklären Sie folgende FreeRtos Semaphore-Funktionen: Was bedeuten die Parameter, was die Rückgabetypen?

- xSemaphoreCreateCounting: Erstellt einen zählenden Semaphore. uxMaxCount legt den maximalen Zählerstand fest. uxInitialCount definiert den Startwert.
- xSemaphoreGive: Gibt den Semaphore frei; erhöht den Zähler. Rückgabe ist pdTRUE bei Erfolg.
- xSemaphoreTake: Reserviert den Semaphore; verringert den Zähler. xTicksToWait definiert die maximale Wartezeit.

5.3 Producer/Consumer Implementierung

Gegeben sind zwei Taskfunktionen, eine für einen Producer und eine für einen Consumer. Der Producer soll abwechselnd alle 30ms bzw 50 ms ein Ereignis produzieren und an den Consumer per Semaphore senden. Der Consumer soll eintreffende Ereignisse per printf() auf die Konsole ausgeben. Implementieren sie auch SetupAndStartTasks() zum Initialisieren und Starten der benötigten Datenstrukturen/Tasks.

```
SemaphoreHandle_t xSem;

void producerTaskFunc(void * pvParameters) {
    for (;;) {
        xSemaphoreGive(xSem);
        vTaskDelay(pdMS_TO_TICKS(30));
        xSemaphoreGive(xSem);
        vTaskDelay(pdMS_TO_TICKS(50));
    }
}

void consumerTaskFunc(void * pvParameters) {
    for (;;) {
        if (xSemaphoreTake(xSem, portMAX_DELAY)) {
            printf("Event verarbeitet\n");
        }
    }
}

void setupAndStartTasks(void) {
    xSem = xSemaphoreCreateCounting(10, 0);
    xTaskCreate(producerTaskFunc, "Prod", 2048, NULL, 2, NULL);
    xTaskCreate(consumerTaskFunc, "Cons", 2048, NULL, 1, NULL);
}
```

5.4 Scheduling Diagramm

Skizzieren Sie das Scheduling in einem Diagramm. Nehmen Sie an, dass eine Zeitscheibe (System Tick) 10ms beträgt und ein IdleTask mit niedrigster Priorität dann Prozessorzeit konsumiert, wenn alle anderen Tasks blockiert sind.

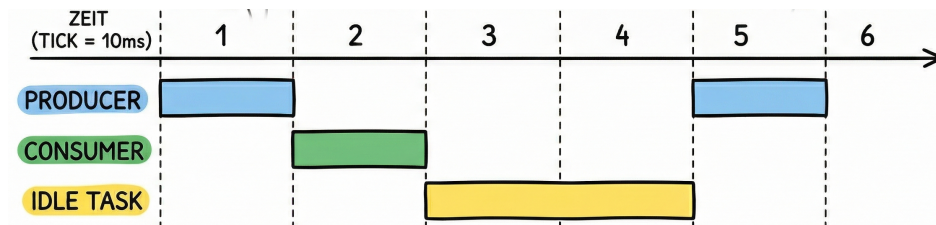


Abbildung 1: Task Scheduling

5.5 Producer/Consumer Erweiterung

Welche anderen (zusätzlich zum Semaphor) Datenstrukturen/Methoden kennen Sie zur Implementierung des Producer/Consumer-Systems?

Sie können folgende Strukturen nutzen:

- Message Queues für den Datenaustausch.
- Stream Buffer für kontinuierliche Daten.
- Task Notifications für einfache Signale.

6 Probeklausur Systemprogrammierung 2024/2025

6.1 Aufgabe 1: Datentypen

1.1 Verwendung von stdint.h

Diese Header definieren Datentypen mit festen Breiten (z. B. `uint32_t`).

Standard-Typen wie `int` oder `long` variieren je nach Architektur. Fest definierte Breiten garantieren die Portierbarkeit und verhindern Überläufe bei Hardware-naher Programmierung.

1.2 Padding

Padding fügt ungenutzte Füll-Bytes zwischen Strukturkomponenten ein. Der Compiler erzwingt damit das Alignment. Es hat Einfluss auf die Speichergröße einer `struct`. Programmierer müssen Padding beachten, wenn sie Strukturen direkt über Netzwerke senden oder binär speichern.

1.3 Alignment

Alignment ist die Ausrichtung von Daten an Speicheradressen, die durch ihre Größe teilbar sind (z. B. 4-Byte-Werte an Adressen wie `0x04`, `0x08`). Padding ist das Werkzeug, um dieses Alignment innerhalb von Strukturen sicherzustellen.

6.2 Aufgabe 2: RGB-Farbe und Union

```
typedef struct {
    uint8_t r;
    uint8_t g;
    uint8_t b;
    uint8_t a;
} RGBColor;

typedef union {
    RGBColor color;
    uint32_t raw;
} ColorUnion;

void printColor(const ColorUnion *c) {
    printf("R=%u G=%u B=%u A=%u, raw=0x%08X\n",
           c->color.r, c->color.g, c->color.b, c->color.a, c->raw);
}
```

6.3 Aufgabe 3: Bitmaskierung

3.1 Bitmaskierung vs. Bitfields

Bitmaskierung nutzt logische Operatoren (&, |, ~). Sie ist explizit und portabel.

Bitfields nutzen die Syntax des Compilers (int a : 1;). Bitfields sind lesbarer, aber die Bit-Reihenfolge ist implementierungsabhängig.

```
// Maskierung
status |= (1 << 3);
// Bitfield
struct { uint8_t flag : 1; } s; s.flag = 1;
```

3.2 Paritätsbit Funktion

```
bool even_parity_bit(uint8_t byte) {
    uint8_t v = byte;
    v ^= v >> 4;
    v ^= v >> 2;
    v ^= v >> 1;
    return (v & 1);
}
```

6.4 Aufgabe 4: Dynamische Speicherverwaltung

```
void mem_get_Sizes(uint32_t* totalAlloc, uint32_t* totalFree) {
    *totalAlloc = 0;
    *totalFree = 0;
    uint32_t offset = 0;

    while (offset < HEAPSIZE) {
        struct Header* h = (struct Header*)&gHeap[offset];
        uint32_t blockSize = sizeof(struct Header) + h->length;

        if (h->flags & 0x01) {
            *totalAlloc += blockSize;
        } else {
            *totalFree += h->length;
            *totalAlloc += sizeof(struct Header);
        }
        offset += blockSize;
    }
}
```

6.5 Aufgabe 5: Multitasking

5.1 Semaphore

Ein Semaphore ist ein Synchronisationsobjekt mit einem Zähler. Er regelt den Zugriff auf Ressourcen oder signalisiert Ereignisse.

- P() (Passeren/Take): Wartet, bis der Zähler > 0 ist, und dekrementiert ihn.
- V() (Vrijgeven/Give): Inkrementiert den Zähler und weckt wartende Tasks.

5.2 FreeRTOS Funktionen

- `xSemaphoreCreateCounting`: Erzeugt Semaphore. `uxMaxCount` ist das Limit, `uxInitialCount` der Startwert. Gibt Handle zurück.
- `xSemaphoreGive`: Gibt Semaphore frei. Rückgabe `pdTRUE` bei Erfolg.
- `xSemaphoreTake`: Belegt Semaphore. `xTicksToWait` ist die Blockierzeit.

5.3 Producer/Consumer Implementierung

```
void ProducerTaskFunc(void * pvParameters) {
    for (;;) {
        xSemaphoreGive(xSem);
        vTaskDelay(pdMS_TO_TICKS(30));
    }
}

void ConsumerTaskFunc(void * pvParameters) {
    for (;;) {
        if (xSemaphoreTake(xSem, portMAX_DELAY)) {
            printf("Ereignis empfangen\n");
        }
    }
}
```

5.4 Producer/Consumer mit Produkten

Für komplexe Datenstrukturen wie `struct Product` ist eine **Message Queue** geeignet. Sie kopiert Daten sicher zwischen Tasks.

```
void ProducerTaskFunc(void * pvParameters) {
    Product p = { .id = 1, .name = "Sensor" };
    for (;;) {
        xQueueSend(productQueue, &p, portMAX_DELAY);
        vTaskDelay(pdMS_TO_TICKS(30));
    }
}

void ConsumerTaskFunc(void * pvParameters) {
    Product p;
    for (;;) {
        if (xQueueReceive(productQueue, &p, portMAX_DELAY)) {
            printf("ID: %d, Name: %s\n", p.id, p.name);
        }
    }
}
```